

ECE379K Lab 1 Report

Part 1:

Why does size() take the lock?

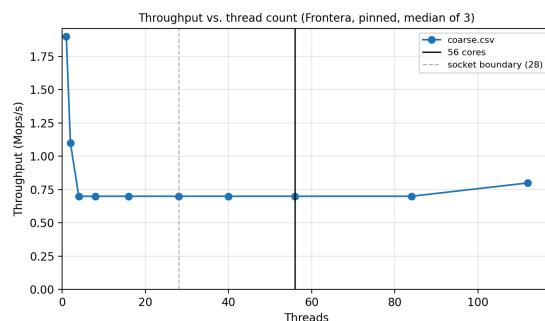
- size() reads the map's element count (nodes in the tree), and insert and erase can modify it. Without a lock, insert/erase could modify the count while size() reads it, causing a data race. By locking size(), the functions become ordered, and we get sequenced behaviour (the lock protects the count data).
- Why does the interface return copies rather than references or iterators?
 - A reference or iterator would last after the lock protects the map (exits once outside of the scope). Theoretically, another thread could erase an entry or add an entry while another thread reads it, creating a data race situation. A copy ensures that the data belongs only to the caller, ensuring thread safety.

Part 2:

- Shape and peak prediction
 - There's only one global mutex, and evidently, from our implementation, every operation requires lock/unlock for the many critical sections (insert, erase, size, find). Therefore, only one thread makes progress at a time – the addition of more threads won't necessarily mean more throughput. We think the addition of more threads will simply ping-pong (the dirty ownership of) the mutex's cache line between threads' core's caches. Therefore, we predict the peak around 1 or 2 threads, and the throughput steadily falling with the addition of more threads. Past 56 threads, we expect it to stay roughly flat or decline slowly.
- Protocol
 - Bench run command: `./bench coarse shards=256 mix=80/10/10 writers=0`
 - All measurements were made on one Frontera compute node (c209-003): two Intel Xeon Platinum 8280 sockets, 28 cores each, 56 cores total, SMT disabled (56 hardware threads). sweep.sh pinned one thread per CPU in CPU-number order (0, 1, 2, ..., 55); for $T > 56$, threads share CPUs 0–55. Each point is the median of three runs; all three are listed in sweep_coarse.log, which you can see below.

```
g++ -std=c++20 -O2 -Wall -Wextra -pthread -o bench bench.cpp
# ./bench coarse shards=256 mix=80/10/10 writers=0
# cores=56 hwthreads=56 sockets=2 cores/socket=28 pin order: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 2
2 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55
T=1      1.9 Mops/s (1.9 1.9 1.9)
T=2      1.1 Mops/s (1.1 1.1 1.1)
T=4      0.7 Mops/s (0.8 0.7 0.7)
T=8      0.7 Mops/s (0.7 0.7 0.7)
T=16     0.7 Mops/s (0.7 0.7 0.7)
T=28     0.7 Mops/s (0.7 0.7 0.7) <- first socket full
T=48     0.7 Mops/s (0.7 0.7 0.7)
T=56     0.7 Mops/s (0.7 0.7 0.7) <- every core busy
T=84     0.7 Mops/s (0.7 0.7 0.7) <- oversubscribed
T=112    0.8 Mops/s (0.7 0.8 0.8) <- oversubscribed
```

- Regarding pin order and socket placement, on this node, lscpu shows CPU numbers are interleaved across sockets. Even-numbered CPUs are on NUMA node 0, and odd-numbered CPUs are on node 1. Then, numeric pin order places consecutive threads on alternating sockets, so every run with more than two threads spans both sockets. The socket-boundary line at 28 marks where the assignment expected the second socket to begin, not where it actually began on this node.



CoarseMap throughput vs. thread count, pinned, median of three. Solid line: core count (56). Dashed line: nominal socket boundary (28).

- Measured vs. Predicted Results + Disclaimer
 - Throughput peaks at T=1 (1.9 Mops/s), as predicted, and falls steeply with the first added threads: to 1.1 Mops/s at T=2 and 0.7 Mops/s at T=4. From T=4 through T=56, throughput is flat at 0.7 Mops/s, and oversubscription past 56 did not reduce it further (0.7 at T=84, 0.8 at T=112), consistent with the prediction.
 - Nothing is visible at T=28. This is expected on this node because CPU numbers interleave sockets; threads were already on both sockets from T=2.

Part 3:

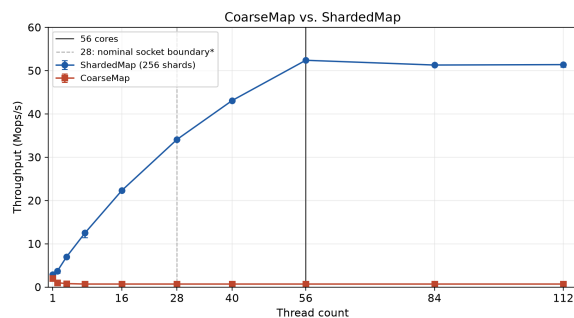
- The line fought over is the one holding the `std::mutex` lock word, which every operation must write actually to acquire the lock. Therefore, the core holding it will have it in the Modified state, and other cores will have an invalid copy. A handoff means snooping the line out of the old holder core (M→I) and into the new one (I→M). Then, similarly, each map line from the old holder (recent inserts, erases, etc.) will go to the new holder. Regarding the cost of handoff, the coherence session measured a same-socket handoff of a Modified line at 12 ns (about 40 cycles) with just two contending cores, rising to 22 ns (around 72 cycles with four), against 14 cycles for an L2 hit and ~74 for L3 (Frontera, 3.29 GHz). Only one thread can be in the map at a time, so extra threads add no throughput, just cost. At T=1, the lock line stays in one core's cache in M, so locking is free. With more threads, every acquisition requires handoff, and handoffs get slower as contenders grow.

T	CPUs	ops (5 s)	Mops/s	cycles/op	instr/op	IPC	L1 miss/op	LLC miss/op	HITM/op	ctx sw/op
1	0	10,523,648	2.10	1,150	259	0.23	18.7	0.35	0.0002	0.0000004
8	0-7 (both sockets)	3,606,528	0.72	14,410	3,671	0.26	98.5	25.4	9.3	0.42
8	even (socket 0)	5,152,768	1.03	9,845	4,128	0.42	114.5	0.78	17.4	0.51
28	0-27 (both sockets)	3,779,584	0.75	74,250	8,659	0.12	115.3	31.9	12.8	0.42
28	even (socket 0)	5,221,376	1.04	51,760	7,910	0.15	147.9	0.59	27.3	0.56

- L1 misses at T=1:** One operation costs 18.7 L1 misses, about one per level of a 19-level tree ($\log_2 512K = 19$). At 64 B per node, the tree is 32 MB, far larger than the 1 MB L2, which holds only the top ~14 levels. So the lower levels miss L1 and L2 but hit the 38.5 MB L3 (only 0.35 LLC misses/op), averaging 62 cycles per miss, which is consistent.
- Cycles vs. misses, T=1 → T=8:** Cycles per op grew $12.5\times$ ($1,150 \rightarrow 14,410$), but L1 misses per op grew only $5.3\times$ ($18.7 \rightarrow 98.5$). At T=1 a miss is a private L2 hit (14 cycles) or L3 hit (74). At T=8, many misses end up finding the line Modified in another core: a handoff costing 40–72+ cycles on one socket (from the coherence session, rising with contenders) and more across sockets.
- HITM:** Measured HITM is 9.3/op with threads on both sockets, and 17.4/op with all 8 on one socket. That's several times the estimate, so most handoffs come from outside the map itself, from possibly the sleep/wake path.
- Context Switches:** At 0.42–0.56 per op, roughly every other operation puts a thread to sleep on the mutex, so it must wake it up later once it's ready to go into the critical section. Threads spend most of their time asleep or on the sleep/wake path, not in the map. Extra threads wait in the kernel rather than competing for the lock line, and so every contended handoff has this cost baked in, causing the “worse rather than flat” phenomenon.

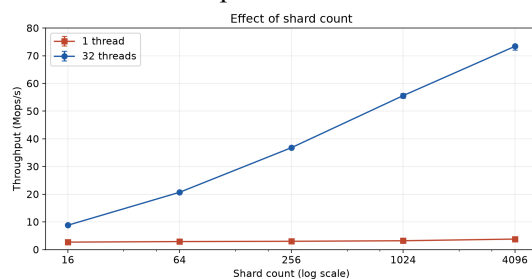
Part 4:

- Correctness checks
 - ShardedMap passed make test and make tsan, including concurrent exact size(), one shard, shared_mutex, and unpadding shards. Both test programs printed “all checks passed” in both runs, with no reported data races.
- Why does index order prevent deadlock?
 - size() acquires every shard’s lock in increasing index order and holds them while summing. Two size() callers follow the same order, so they cannot form a circular wait. Against insert(), size() may wait, but insert() only holds one lock and requests no others. It finishes and releases that lock, allowing size() to continue.
- What could an approximate size() return?
 - Releasing each lock before taking the next combines counts from different times. Suppose A has one key and B is empty. Count A, then another thread erases that key and inserts one into B. Counting B gives a total of 2, although the actual total only went $1 \rightarrow 0 \rightarrow 1$. Holding every lock prevents this.
- Protocol
 - Measured on Frontera node c211-020 using GCC 13.2.0, two Xeon Platinum 8280 sockets, 28 cores each, 56 total, SMT disabled. Used sweep.sh, the 80/10/10 mix, two-second timed runs, and median of three. Pin order was 0–55; even CPUs are socket 0 and odd CPUs socket 1, so both sockets are used from T=2.



CoarseMap vs. ShardedMap (256 shards), pinned, median of three. Solid line: 56 cores. Dashed: nominal socket boundary (28), not the actual transition. Error bars: min–max of three runs.

- Measured Results
 - Sharded throughput rises from 2.9 Mops/s at T=1 to 52.4 at T=56 ($18.1\times$), versus 0.7 for the coarse map at T=56 (about $75\times$). Past 56 threads, it stays near 51 Mops/s. Independent shards allow concurrent operations, although collisions and coherence costs still limit scaling.
- Shard-count Sweep



Shard-count comparison at T=1 and T=32, pinned, median of three. Logarithmic shard-count axis; error bars show min–max of three runs.

- For $N = 16, 64, 256, 1024$, and 4096 , one-thread throughput was 2.7, 2.9, 3.0, 3.2, and 3.8 Mops/s; at 32 threads, it was 8.8, 20.7, 36.8, 55.6, and 73.4. The $1.4\times$ one-thread improvement reflects smaller trees and other locality effects without contention. The $8.3\times$ improvement at 32 threads is consistent with also reducing contention.
- Collision Argument + Shard-count Choice
 - Assuming uniform, independent shard choices, two operations collide with probability $1/N$. For one operation among T simultaneous operations, $p = 1 - (1 - 1/N)^{(T-1)}$. At T=32, p is 86.48%, 38.63%,

11.43%, 2.98%, and 0.75% for the five shard counts. These are modeled overlapping choices, not measured lock-wait probabilities.

- Using throughput X in Mops/s, uncontended time $u = 1000/X_1$ ns gives 370, 345, 333, 313, and 263 ns/op. The per-thread time proxy $v = 32,000/X_{32}$ assumes roughly equal progress. In the simple model $v \approx u + pC$, at $N=256$, $v=870$ ns and $C \approx 4.69$ μ s; $pC \approx 536$ ns is $1.61 \times u$. At $N=4096$, $v=436$ ns and $C \approx 22.9$ μ s; $pC \approx 173$ ns is $0.66 \times u$.
- C is an effective penalty inferred by assigning all excess time to collisions, not a directly measured mutex wait. Cache, scheduling, and other overhead also contribute; dividing them by a small p inflates C . Therefore, these figures quantify the model but do not establish a fixed cost per actual collision.
- We would choose 4096 shards among those tested: it has the best throughput at both thread counts and modeled collision probability below 1% at $T=32$. Its 32-thread runs span 72.1–73.6 Mops/s, versus 54.7–56.4 for 1024, so the gain exceeds observed variation. We did not test larger N and do not claim a global optimum.

Part 5:

Correctness checks

Restored make test/tsan: map and lock programs printed “all checks passed,” exit 0. TSan used Lonestar 6, GCC 15.2.0, CPUs 0–1. Unrestricted TAS runs timed out; affinity alone allowed completion with unchanged code/iterations. Cause unproven.

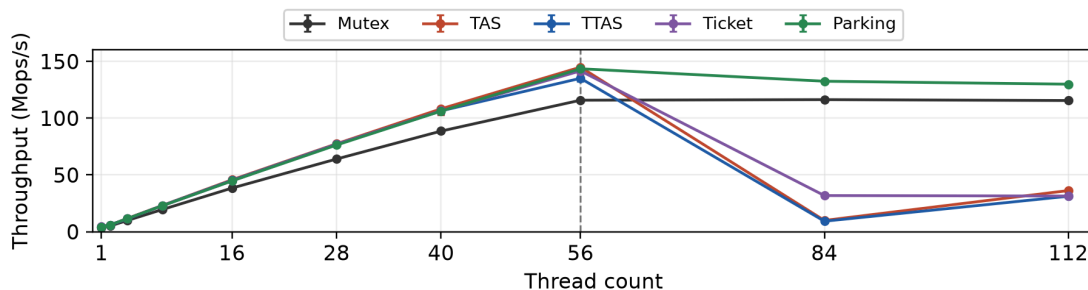
Relaxed-ordering Experiment

Only TASLock’s exchange/store became relaxed. Normal tests passed (0); TSan found a race in `ShardedMap<TASLock>::insert, concurrent_map.h:128` (error 66; make exit 2). Relaxed keeps the flag atomic but removes acquire/release’s happens-before relation for map accesses. x86 hardware ordering can hide this from outcome checks.

Protocol

Frontera c210-036, GCC 13.2.0, -O2, 56 cores/two sockets, 80/10/10 mix. Sweeps: $N=4096$, median of three two-second runs, CPUs 0–55 alternating sockets from $T=2$. Counters: CPUs 0,2,...,14 (one socket), 30 seconds, 250 ms delay.

Plot



4096 shards; min–max bars; dashed: 56 cores. No socket transition at 28.

Parking vs. Mutex

Parking reached 143.5 vs. 115.8 Mops/s at $T=56$ and 129.9 vs. 115.5 at $T=112$, exceeding trial spread.

Fast CAS/bounded spinning can avoid short sleeps; parking limits oversubscribed spinning.

One-shard Counters

Prediction: TAS most L1 misses (ownership traffic), Ticket most instructions (poll/yield). Instead, mutex led both. Counts include program/kernel work, not just the lock.

Lock	Operations	Mops/s	Cycles/op	Instr./op	L1 miss/op	HITM/op	CS/op
Mutex	31,968,256	1.065	12,469	5,264	146.38	22.626	0.653
TAS	35,742,720	1.191	22,027	411	54.49	3.324	3.27e-06
TTAS	65,030,144	2.167	12,106	1,071	38.89	1.516	2.05e-06
Ticket	43,139,072	1.438	18,253	4,118	48.93	2.583	5.49e-06

Parking	39,050,240	1.302	18,482	4,130	93.85	13.380	0.0837
---------	------------	-------	--------	-------	-------	--------	--------

T=8, N=1; CS = context switches. Delay omits ~0.4% of timed work.

TAS vs. TTAS

TAS exchanges repeatedly obtain exclusive ownership and invalidate other copies. TTAS waiters read shared copies; release invalidates them, then contenders race. Backoff reduces attempts. Its lower L1/HITM counts support reduced traffic, but demand-load counters do not capture every atomic ownership transfer.

Oversubscription Counters

Lock	Operations	Mops/s	Cycles/op	CS/op	Busy ¹	Busy ²
Mutex	766,404,608	25.538	1,013	0.000126	98.0%	98.0%
TAS	19,299,328	0.625	41,472	0.001545	98.2%	98.2%
TTAS	22,893,568	0.734	35,066	0.001282	97.5%	97.5%
Ticket	276,854,784	9.180	2,853	0.3748	99.2%	99.2%
Parking	870,614,016	29.014	890	0.0003204	97.8%	97.8%

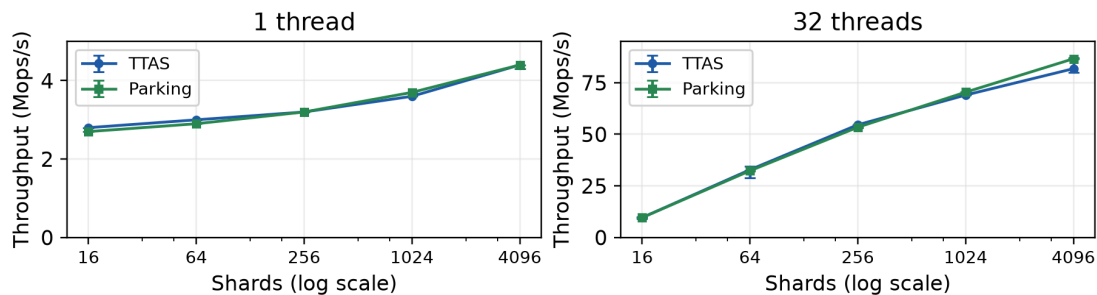
T=32, N=4096, eight cores. Rates use exact operations/elapsed time, including worker completion.

Busy¹ = (cycles/op × Mops/s × 10⁶)/(8 × 3.30 × 10⁹); cycles/task-clock gives 3.30 GHz, above nominal 2.70. Busy² = task-clock/(8 × elapsed), both in ms. Delayed counting omits about 0.17 s, so ratios are approximate.

What happens beyond the core count?

CPUs stayed 98–99% busy, but TAS/TTAS spun while owners could be descheduled, making little progress. Ticket yielded (0.3748 switches/op), yet FIFO still awaited specific owners. Parking/mutex let waiters sleep while runnable threads progressed; sleeping waiters did not leave CPUs idle.

Shard-count Sweep + Collision Argument



Median of three, min–max bars; Part 4 pinning. T=1 isolates locality gains.

For independent uniform choices at T=32, $p = 1 - (1 - 1/N)^{31}$: 86.48%, 38.63%, 11.43%, 2.98%, 0.75% for the plotted N. With X in Mops/s, $u=1000/X_1$ ns, $v=32,000/X_{32}$ ns, $v \approx u + pC$. C absorbs other scaling overhead; it is not measured collision latency.

Lock	N	u (ns)	v (ns)	C (μs)	pC (ns)
TTAS	256	312.5	586.1	2.39	273.6
TTAS	4096	227.3	391.2	21.74	163.9
Parking	256	312.5	599.3	2.51	286.8
Parking	4096	227.3	369.5	18.86	142.2

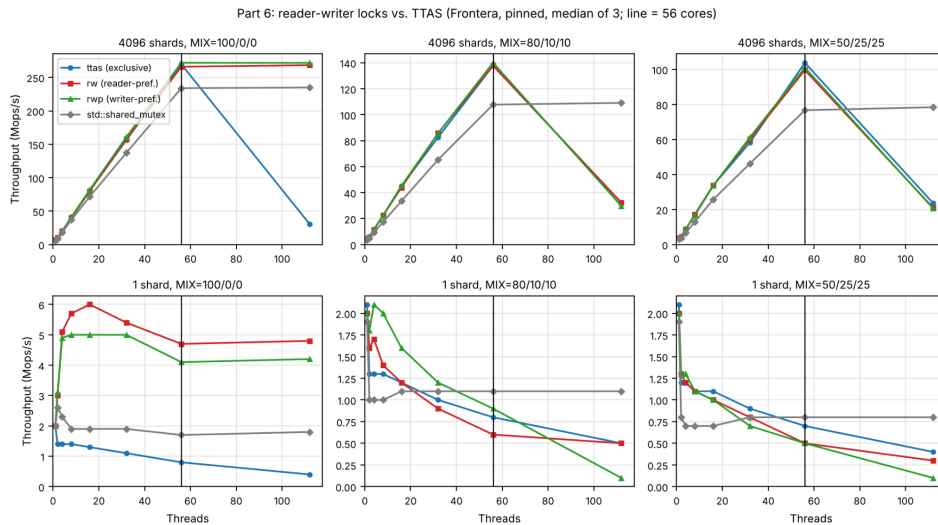
Keep N=4096: pC/u is 0.72 (TTAS) and 0.63 (Parking), versus 0.88/0.92 at N=256. T=32 ranges of 80.1–87.1/86.4–86.7 Mops/s exceed N=1024's 68.9–69.0/70.4–70.5. It is best among tested counts, not a proven global optimum.

Spin Hint + Ticket Yield

Pause reduces aggressive spinning without giving up the timeslice. Ticket yields because FIFO requires the exact next owner to run. TTAS allows any running contender to acquire a free lock, but still suffers if the current owner is descheduled.

Part 6:

- Correctness: RWLock and RWLockWP passed make test and make tsan, both as exclusive locks (8 threads, mutual exclusion) and in the 6-reader + 2-writer sharing test. We verified that we got “all checks passed”.



All 24 Part 6 configurations (four locks $\times$ three mixes $\times$ two shard counts): throughput vs. threads, pinned, median of three. Rows are $N = 4096$ and $N = 1$; columns are the mixes 100/0/0, 80/10/10, and 50/25/25. The solid line is 56 cores.

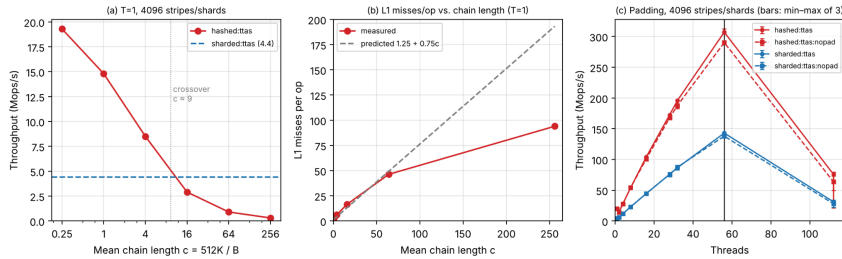
- When does the RW lock beat TTAS?
 - One shard but tread-heavy: We saw at 100/0/0, RW reaches 6.0 Mops/s at $T=16$ vs. 1.3 for TTAS (4.6 $\times$), and 4.8 vs. 0.4 at $T=112$. At 80/10/10 the gain is smaller but still there to some extent: 1.7 vs. 1.3 at $T=4$, but no longer holds by $T=32$ (0.9 vs. 1.0).
 - Another case we saw was for many shards, read-only, and oversubscribed. At $N = 4096$, 100/0/0, Threads=112 is 268 vs. 31 Mops/s.
 - Everywhere else RW ties or loses. At $N = 4096$ and $T \leq 56$ the spinning locks are within run-to-run spread of each other (80/10/10, $T=56$: 137.8 vs. 137.8), and at 50/25/25, RW is slightly worse (99.6 vs. 103.8). A reader pays what a TTAS acquire pays: one atomic RMW to enter (CAS $n \rightarrow n+1$) and one to leave (fetch_sub). It gains only when it would otherwise have waited behind another reader.
- Std::shared_mutex
 - This is the slowest under low contention: 3.5 vs. 4.3 Mops/s at $T=1$ (−19%) and 107.8 vs. 137.8 at $T=56$ ($N = 4096$, 80/10/10), and we think it’s because its entry and exit cost more. It also doesn’t collapse past 56 threads (109 vs. ~ 32 at $T=112$), because its waiting threads just sleep instead of spinning.
 - At one shard shared_mutex doesn’t scale reads (1.9 Mops/s). The writers run shows how it treats a waiting writer. With 1–2 writers, writers get about 0 while readers get 2. New readers keep entering while a writer waits, which is reader-preferring. With 4 or more writers it flips (one $W=4$ run, and all $W=8$ runs): readers get 0 and writers get 0.6–0.8. Whichever class holds the lock tends to keep it.
- Writer pref table

Writers	ttas	rw	rwp	shared_mutex
1	1.3 / 0.0 / 1.3	5.2 / 0.0 / 5.2	0.1 / 1.0 / 1.1	2.0 / 0.0 / 2.0
2	1.3 / 0.0 / 1.3	4.8 / 0.0 / 4.8	0.0 / 0.7 / 0.7	1.8 / 0.0 / 1.9
4	1.2 / 0.0 / 1.2	3.8 / 0.0 / 3.8	0.0 / 0.6 / 0.6	1.9 / 0.0 / 1.9
8	1.1 / 0.1 / 1.2	1.7 / 0.4 / 2.1	0.0 / 0.6 / 0.6	0.0 / 0.6 / 0.7

RW starves its writers until $W=8$. However, writer preference reverses this: with one writer, writers rise from 0 to 1.0 Mops/s, readers fall from 5.2 to 0.1, and the total falls from 5.2 to 1.1 (−79%). Each writer re-announces itself as soon as it unlocks, so the waiting-writer count is almost never zero. Readers that could have shared the lock wait instead, and the lock runs at practically the serialized-write rate.

Part 7:

- For mapping, Bucket b belongs to stripe $\lfloor b \cdot L/B \rfloor$, where B is # of buckets and L is # of stripes, so each stripe guards a contiguous range of buckets. Neighbouring stripes share only one boundary line of bucket heads, which is negligible. The sharing that does matter is the stripe locks themselves when unpadded (see Padding).



1. Bucket Count Crossover

B	c	hashed Mops/s	L1 miss/op	cycles/op	Report Explanation
2M	0.25	19.3	2.4	132	Crossover at c around length 9 (Buckets $\approx 56K$). A tree lookup visits about 7 nodes (128 keys per shard); a hash lookup visits about $0.75c$.
512K	1	14.8	3.1	175	For misses, we predicted $1.25 + 0.75c$. Measured slope 0.67 for $c = 4-64$, graph it flattens at $c = 256$ (see above).
128K	4	8.5	6.2	307	Per-miss cost: 3629 extra cycles / 47.7 extra misses which is about 76 cycles, an L3 hit (≈ 74 in the coherence session). The table is too big for L2 but fits in L3.
32K	16	2.9	16.6	868	
8K	64	0.9	46.3	2873	
2K	256	0.3	94.0	6502	

2: Stripe Count

L	ttas	park	P(collision)	Explanation
16	9.1	10.2	86%	An uncontended operation costs ≈ 51 ns. Therefore, one collision costs $\approx 0.8-1.5$ μ s (15–30 operations).
64	45.1	35.8	39%	Ship $L = 4096$: collision cost ≈ 6 ns per operation, vs ≈ 24 ns at 1024. Its runs (193–195) sit above $L = 1024$'s (169–184).
256	108.2	103.7	11%	
1024	176.7	166.8	3.0%	
4096	194.7	184.6	0.75%	

3: Padding

28 threads, socket 0, 1024 stripes	ops (5 s)	Mops/s	cycles/op	L1 miss/op	HITM/op
padded	1,416,913,920	283.3	230.7	3.39	0.795
unpadded	1,013,492,736	202.5	322.5	3.65	0.802

Prediction (before the run): 16 B per stripe, so 4 locks per cache line, so we expected an effect only at high T.

Sweep, 4096 stripes: the largest gap is at T=56: hashed -5.5% (307.0 vs 290.1), sharded -3.5%. There's no difference beyond noise at $T \leq 16$. This could be bounded by how often a neighbouring stripe on the same line is busy.

Throughput shows -29%: HITM shows no change. The false sharing comes from stores (TTAS's exchange and release store), and HITM counts loads only.

Takeaway: the lock is cheap while its cache line stays private, but sharing that line adds about 92 cycles per operation, and the load-based counters can't see it.

Appendix

Appendix A, Contributions: Connor implemented ShardedMap (Part 4) and the four Part 5 locks, ran their measurements, and wrote the Part 4 and Part 5 sections. Krish implemented CoarseMap (Part 1), the reader-writer locks (Part 6) and StripedHashMap (Part 7). Krish also ran the Part 2, 3, 6, and 7 measurements and wrote those sections. Connor helped with Parts 6 and 7. We reviewed each other's code and results, and both of us can explain every part.

Appendix B, Machine + Protocol: All measurements: one Frontera compute node, $2 \times$ Intel Xeon Platinum 8280, 28 cores per socket, 56 cores, SMT off. sweep.sh pinned one thread per CPU in CPU-number order (0, 1, ..., 55); every point is the median of three runs. [sweep.sh](#) boundary is at $T = 28$, but on these nodes CPU numbers alternate sockets (below), so both sockets are in use from $T = 2$.

lscpu:

```
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Byte Order: Little Endian
CPU(s): 56
On-line CPU(s) list: 0-55
Thread(s) per core: 1
Core(s) per socket: 28
Socket(s): 2
NUMA node(s): 2
Vendor ID: GenuineIntel
CPU Family: 6
Model: 85
Model name: Intel(R) Xeon(R) Platinum 8280 CPU @ 2.70GHz
Stepping: 7
CPU MHz: 3299.853
CPU max MHz: 4000.0000
CPU min MHz: 1000.0000
bogomips: 5400.00
Virtualization: VT-x
L1d cache: 32K
L1i cache: 32K
L2 cache: 1024K
L3 cache: 39424K
NUMA node0 CPU(s): 0,2,4,6,8,10,12,14,16,18,20,22,24,26,28,30,32,34,36,38,40,42,44,46,48,50,52,54
NUMA node1 CPU(s): 1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39,41,43,45,47,49,51,53,55
```

Appendix C: TSan (ECE LRC mario, GCC 13.3, all HAVE_* = 1, halt_on_error=1)

- Test_map_tsan and test_locks_tsan both pass

```
ks57656@mario (/home/eceirc/students/ks57656/ECE379K/lab1/starter_files) % grep -c 'WARNING: Thread
1.log # should print 0
grep: tsan_full.log: No such file or directory
ks57656@mario (/home/eceirc/students/ks57656/ECE379K/lab1/starter_files) % make test_map_tsan && ./
1 -3 map_tsan | tail
make: 'test_map_tsan' is up to date.
StripedHashMap<RWMP,nopad>: concurrent size(), exact
StripedHashMap<RWMP,nopad>: 0.1 s
all checks passed
```